

# FINAL YEAR CS PROJECT – POC PRESENTATION

## AI Gaming E-Commerce Application

### Team BIT-BY-BIT

By: Joshua Delshad, Fazlul Faizal, Moh Prajapati, Fotios Bampouridis, Eitan Abrishami

# Bit-By-Bit – Fully Functional Phase 2 System

## Project Introduction

BitByBit is a gaming e-commerce platform that allows users to browse, search, and purchase games, consoles, and accessories through a mobile application. The project integrates React Native, Django APIs, MongoDB, and the RAWG API to provide real-time game data and a smooth shopping experience. In Phase 2, the system was expanded with checkout functionality, AI-powered recommendations, and cloud-ready API integration.

## Team Members and Instructor details:

- Fazlul H Faizal: Backend Developer, System Architecture Design, AI Integrator
- Moh Prajapati: Document Creator/ Frontend Developer
- Joshua Delshad: Backend, LEADER, Database Developer
- Fotios Bampouridis: Frontend Developer
- Eitan Abrishami: Backend, Database, Frontend Developer

## Course :

- COMP 491/L

Logo:

The logo for GameStart, featuring the word "GameStart" in a bold, cyan-colored font with a slight 3D effect, set against a dark background with a thin blue horizontal line below it.



# Problem Statement

## Core Problem Description

- Users visit multiple platforms for stats, reviews, and pricing. There are no gaming ecommerce applications that leverage AI including AI chatbot and AI recommendations. No single hub exists for real-time game intelligence.

## Affected Stakeholders

- Gamers, retailers, and developers are impacted by fragmented platforms, lack of AI features, and limited product visibility

## Pain Points and Gaps

- Users don't really have a bridge to find the stats on games so they visit youtube or other websites like IGN. But with our application users can get latest updates using our RAWG API and Groq Llama AI model.

## Visual Problem Flow

- Diagram illustrating the transition from current state to identified pain points, enhancing audience understanding.



# Objective / Goals

## Enhance Full System Functionality

- Expand the platform into a fully connected full-stack gaming e-commerce application.
- Integrate frontend, backend APIs, MongoDB database, and cloud-ready services.

## Implement Advanced Features

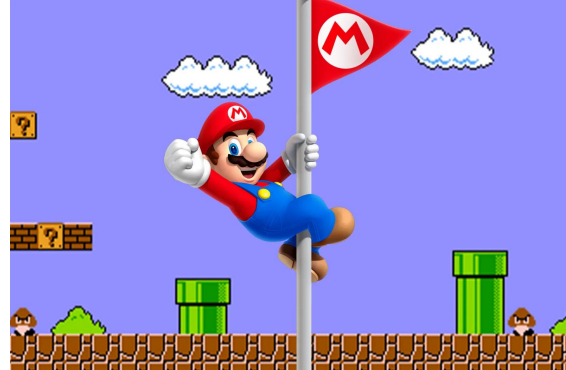
- Support secure checkout flow, dynamic cart updates, and API-based product management.
- Add AI-powered recommendations and live game data using the RAWG API.

## Improve User Experience

- Enhance category filtering, navigation flow, and responsive UI design.
- Ensure smooth interaction between Home → Search → Cart → Checkout.

## Success Criteria Checklist

- Users can successfully browse, search, add products to cart, and complete checkout.
- APIs, database integration, AI recommendations, and frontend features operate reliably and efficiently.





# Architecture Overview

## System Components

Frontend/UI: React Native mobile application with category browsing, cart management, checkout, and user navigation.

Backend/API Layer: Django REST APIs handle authentication, cart operations, product management, and checkout processing.

Database Layer: MongoDB stores user accounts, cart data, order history, and product information.

External Services / AI: RAWG API provides live game data, ratings, and metadata, while AI-based recommendations improve product discovery.

## Data Flow Representation

User → React Native Frontend → Django APIs → MongoDB Database / RAWG API → Response Back to App

Arrows show how requests move from the app to the backend and return processed data to the UI.

## Technology Stack Legend

Frontend: React Native

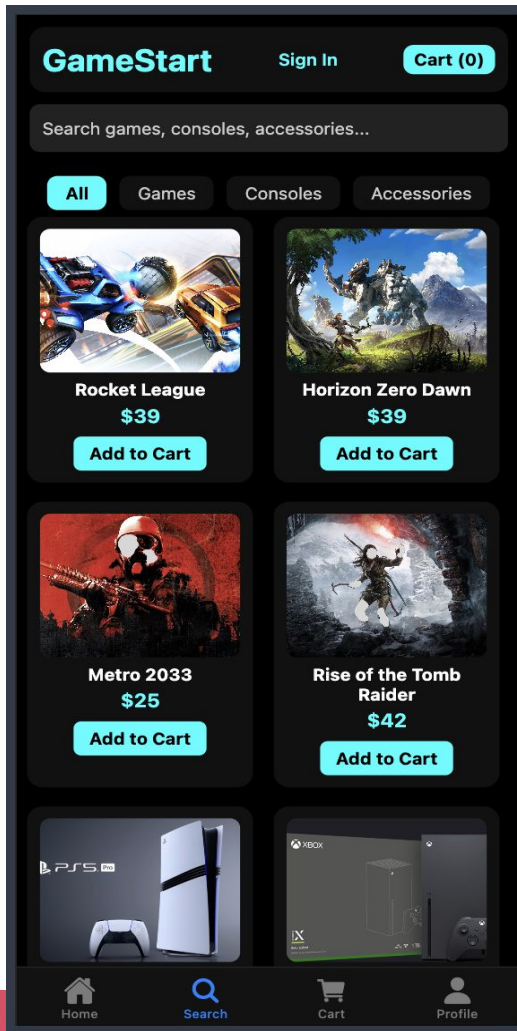
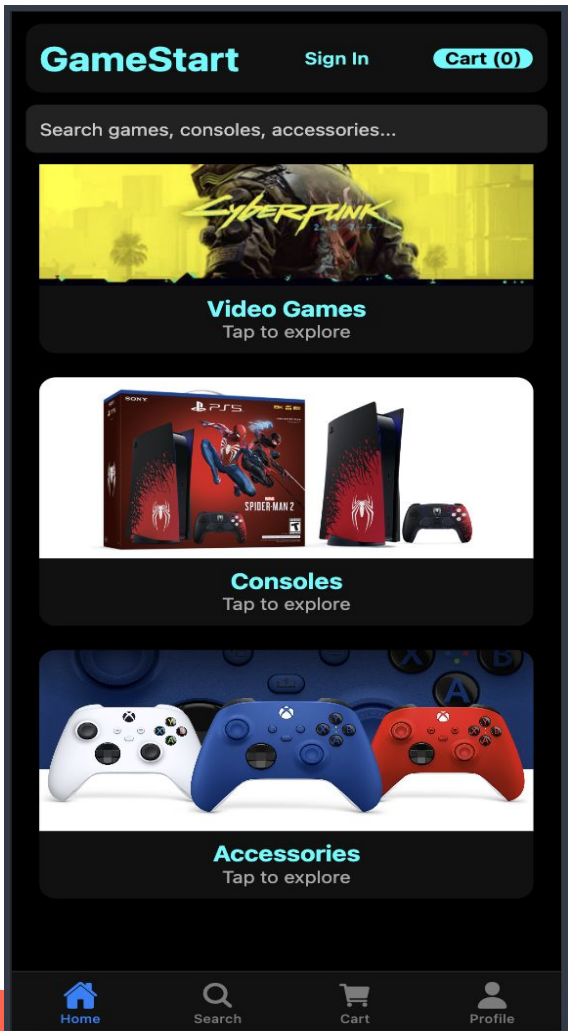
Backend: Django REST Framework

External Services: MongoDB Atlas

Database: RAWG API

Cloud services: Azure App Service





# Design & Mockups

## Visual Design Overview

Displays core app screens (Home, Search, Cart, Profile) to show interface layout and navigation flow. Highlights the gaming-inspired dark theme

## User Experience Considerations

Designed for **minimal clicks** and clear navigation using bottom tabs. High-contrast colors and large buttons improve readability and touch usability.

## User-Centered Design Focus

Layout and interactions prioritize how gamers explore products and manage carts.

Screens and actions reflect real user behaviors, making the app intuitive and familiar.

GameStart

Sign In

Cart (0)

Search games, consoles, accessories...

All

Games

Consoles

Accessories



PlayStation 5  
\$499.99

Add to Cart



Xbox Series X  
\$499.99

Add to Cart



Nintendo Switch  
\$299.99

Add to Cart



Home



Search



Cart



Profile

Your Profile



Guest User

guest@gamestart.com



My Orders



↪ Sign Out



Home



Search



Cart



Profile

Your Cart

**Your cart is empty**

Add some games to get started!



Home



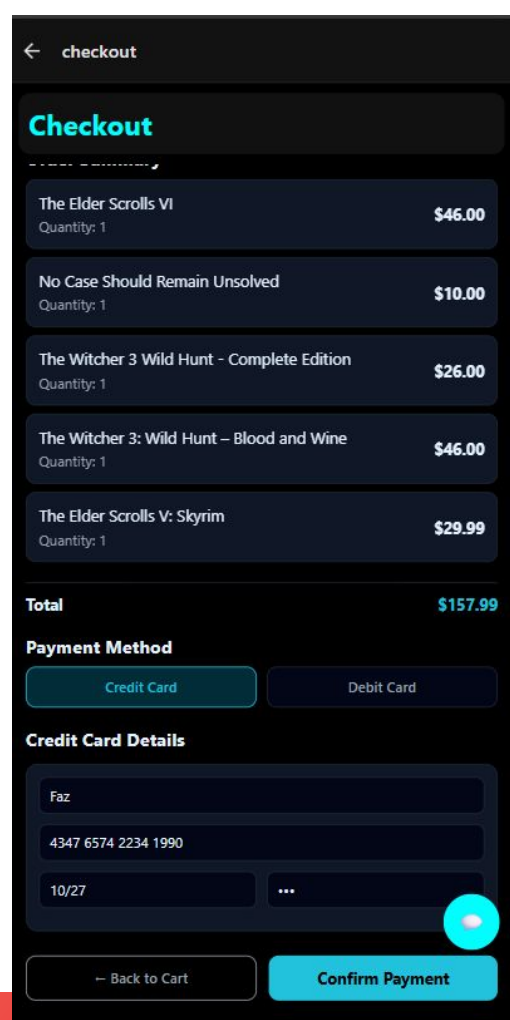
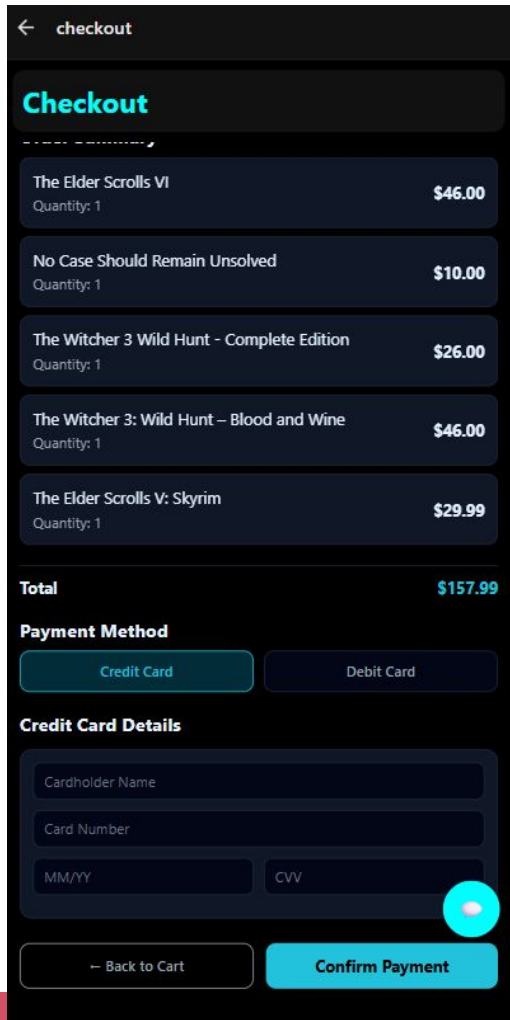
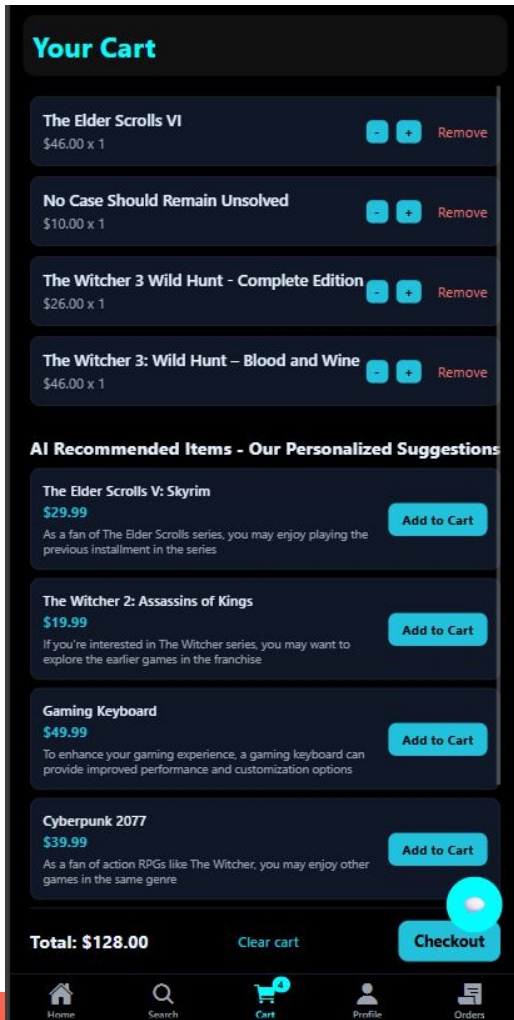
Search



Cart



Profile





← checkout

## Payment Successful



**Payment Confirmed!**

**Total: \$157.99**

Order Status

**Processing**

Cardholder

**Faz**

Card

.....1990

Continue Shopping

View Orders

## Order History

**Order #6a04005c6d0b2ea628b848ad**

May 12, 2026, 9:38 PM PDT

**Total: \$157.99**

Status: confirmed

- The Elder Scrolls VI x1 @ \$46
- No Case Should Remain Unsolved x1 @ \$10
- The Witcher 3 Wild Hunt - Complete Edition x1 @ \$26
- The Witcher 3: Wild Hunt – Blood and Wine x1 @ \$46
- The Elder Scrolls V: Skyrim x1 @ \$29.99

← Back



## GameStart Assistant

Ask about products, carts, and deals



Hi Faz, I'm your GameStart AI assistant. I can help you find games, compare consoles, and think through accessories. What are you looking for today?

Ask about games, consoles, accessc

Send

# Cloud API Example 1: Submit Order API

**API Name:** Submit Order API

**Purpose:** Submit Orders from the frontend to the backend to go to the Azure Cloud-hosted Database

**Method:** POST

**Input:** {

    "userEmail": user\_email,

    "items": [

        {

            "name": items[i],

            "quantity": quantities[i],

            "unitPrice": str(unit\_prices[i]),

        }

    }

**output:** {

    "success": success status,

    "order\_id": mongo\_id,

    "email": user\_email,

    "date": timezone.now().isoformat(),

    "total": str(total\_price),

    "email\_sent": email\_sent,

    "message": message,

}

# Cont. Cloud API Example 1: Submit Order API

**API Name:** Submit Order API

**Purpose:** Submit orders from the frontend to the backend to go to the Database that is hosted on Azure Cloud

**Method:** POST

## JSON EXAMPLE

**Input:** {

    "userEmail": example@gmail.com,

    "items": [

        { "name": God of War,

          "quantity": 2,

          "unitPrice": 50,

        }, { "Name": Gaming Mouse,

            "quantity": 1,

            "unitPrice": 30, }]

}

**output:** {

    "success": True,

    "Order\_id": 23uwqe818ueq82uq12ew2ee,

    "email": example@gmail.com,

    "date": may 14, 12:30 pm,

    "Total": 130,

    "email\_sent": example@gmail.com

    "message": "Order submitted successfully",

}

# API Example 2: AI Recommendation API

**API Name:** AI Recommendations API using Grok LLM

**Purpose:** Based on the users selected products the AI recommends similar items

**Method:** POST

**Input:** { **JSON Template**

```
"Items": [  
  {"id": 123, "name": "Item Name", "price": 29.99, "quantity": 2},  
  ... ] }
```

**JSON EXAMPLE**

**output:** {

```
"Success": status Status,  
  
  Recommended items: {"id": 1, "name":  
    "Recommended Item 1", "price": 39.99, "reason":  
    "Often paired with items in your cart"},  
  
  }
```

**output:** {

**"Success": status Status**

**"Recommended Items:**

```
  {"id": 223, "name": Assassins Creed  
    Brother Hood, "price": 50 , "reason": related to  
    item in cart}, {"id": 223, "name": Gaming Mouse,  
    "price": 15, "reason": related to item in cart}, }
```

**Input:** {

```
"Items": [  
  {"id": 123213, "Name": Assassins Creed II, "price": 50, "quantity": 2},  
  {"id": 1212, "Name": Grand Theft Auto 5, "price": 49, "quantity": 2},  
  ... ] }
```

| Phase                                                            | Objective                                           | Key Tasks / Activities                                                                                 | Testing & Validation                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1: Foundation Building & Core Functionality Implementation | Establish core infrastructure                       | Web framework setup, database creation, product categories, basic UI prototype                         | <b>Comprehensive Test Cases</b> <ul style="list-style-type: none"> <li>Tested AI recommendation responses and API integrations.</li> <li>Validated login, cart operations, checkout flow, and category filtering.</li> <li>Verified frontend/backend communication and MongoDB data storage.</li> <li>Tested unauthorized access protection and API reliability.</li> </ul> |
|                                                                  | Deploy essential features and systems               | Login tests, navigation & cart checks, performance metrics, bug tracking & fixes                       |                                                                                                                                                                                                                                                                                                                                                                             |
| Phase 2: Core Functionality Implementation                       | Integrate AI-powered recommendations and cloud APIs | AI recommendation setup, RAWG API integration, backend/frontend API communication, MongoDB integration | <b>Quality Assurance Evidence</b> <ul style="list-style-type: none"> <li>Screenshots, API responses, and testing logs confirmed stable system behavior.</li> <li>Peer testing helped improve usability, navigation flow, and checkout experience.</li> </ul>                                                                                                                |
|                                                                  | Improve personalized shopping and product discovery | Semantic search, category filtering, dynamic recommendations, responsive UI improvements               |                                                                                                                                                                                                                                                                                                                                                                             |
|                                                                  | Ensure reliable API and database performance        | Checkout API testing, authentication validation, cart synchronization, cloud API testing               | <b>Issue Tracking and Limitations</b> <ul style="list-style-type: none"> <li>Improved API response speed and frontend performance.</li> <li>Reduced UI clutter and enhanced checkout workflow.</li> <li>Improved AI recommendation accuracy and product search experience.</li> </ul>                                                                                       |
|                                                                  | Optimize and maintain full system functionality     | Bug fixes, API response optimization, user feedback testing, cloud deployment preparation              |                                                                                                                                                                                                                                                                                                                                                                             |



# Legal, Security & Cloud Compliance



## Data Security & Privacy

- Secure user authentication and protected API access using backend validation.
- Store user, cart, and order data securely in MongoDB Atlas cloud database.

## API & Cloud Security

- Cloud-ready APIs deployed through Azure App Service architecture.
- Secure communication between React Native frontend and Django backend APIs.

## Authentication & Authorization

- Support secure login, user session management, and protected checkout operations.
- Restrict unauthorized access to sensitive user and order data.

## Third-Party & AI Compliance

- Follow RAWG API usage policies and external data licensing requirements.
- Ensure AI-powered recommendations and third-party assets comply with platform guidelines.

# Demo Flow

## Setup:

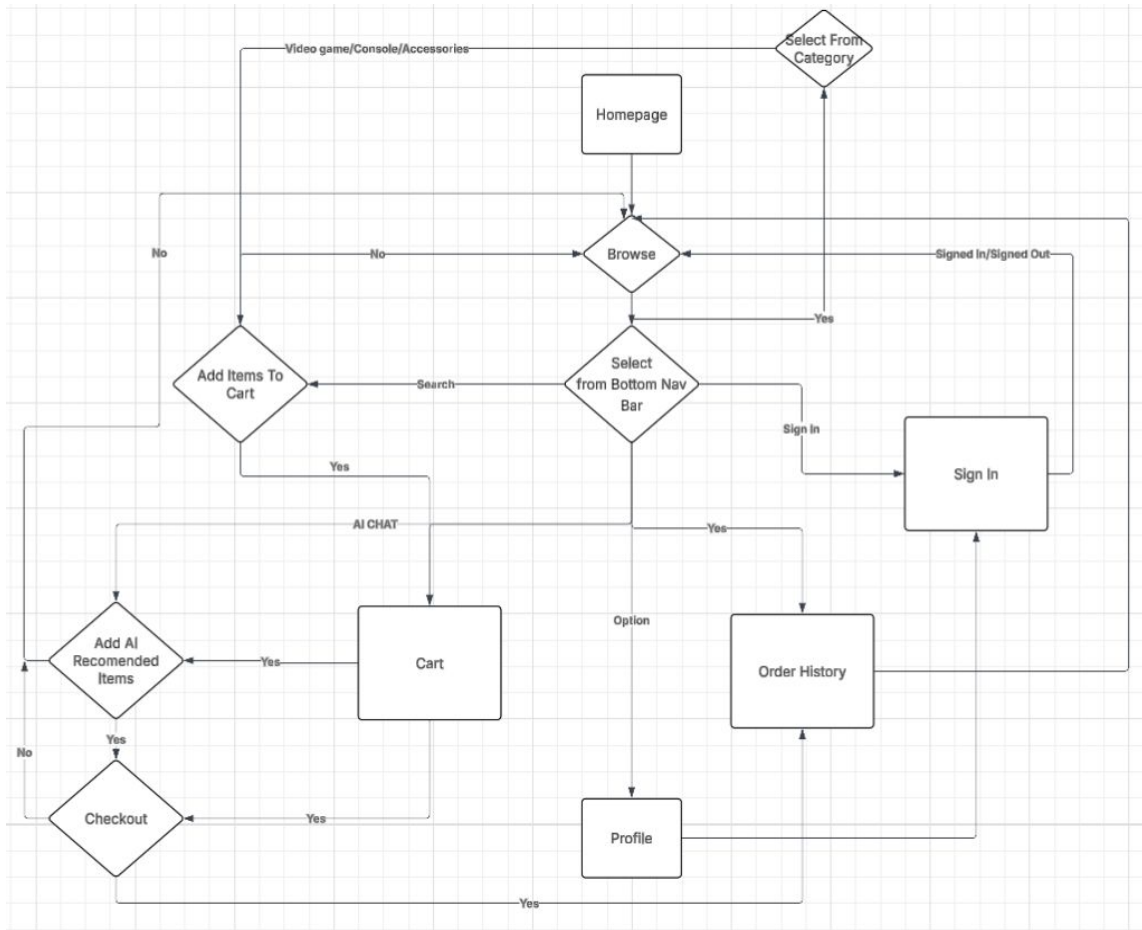
- Open app & show main tabs
- Display AI Chat Icon

## Action:

- Select Category from Home
- search
- add to cart
- Use Ai Chat
- Select AI Recommended games

## Result:

- View cart with selected items
- Complete Order
- G



# Feedback & Iteration

## AI & User Feedback Collection

- Collected feedback from peer testing to improve AI recommendations, frontend usability, and API integration.

## Frontend & UI Improvements

- Improved navigation flow and reduced UI clutter for a smoother shopping experience.
- Enhanced category filtering and cart interaction using dynamic frontend components.

## API & System Optimization

- Optimized Django API communication for faster response times and smoother frontend/backend interaction.
- Improved MongoDB query handling and checkout API performance.

## AI Recommendation Enhancements

- Refined AI-powered recommendation logic and product discovery experience.
- Refined AI-powered Chat to promote user experience by providing immediate feedback

## Visual Before-and-After

- Displayed UI and feature improvements between Phase 1 and Phase 2 implementation.



# Metrics & Achievements

## Full-Stack System Performance

- Improved frontend/backend communication and API reliability across the application.
- Successfully integrated React Native frontend with Django APIs and MongoDB database.

## AI & API Response Optimization

- Reduced API response time and improved dynamic product loading performance.
- Enhanced AI-powered recommendation speed and product search accuracy.

## Feature Completion & Integration

- Successfully implemented checkout flow, cart synchronization, category filtering, and cloud-ready APIs.
- Integrated RAWG API for live game data, ratings, and metadata support.

## Cloud & Database Reliability

- Achieved stable MongoDB data storage for users, carts, and order management.
- Validated cloud-ready backend architecture and secure API communication.

## Positive User & Peer Feedback

- Users reported smoother navigation, responsive UI, and improved shopping experience.
- Peer testing confirmed improved usability, checkout flow, and frontend performance.



# Challenges/What We Learned

## Challenges

- Problem: the Orders API and AI APIs were not connecting kept on showing “networked failure” or > failure
- Solutions: the URLs needed to be updated to use the Azure URL or the backend needed to be deployed to Azure Cloud or the API keys expires
- Problem: Email API failed work on the main branch while working on local branch
- Solution: Needed to resolve many merge conflicts properly and integrate the proper code and Email API info inside the ENV file while deploying the info to Cloud
- Problem: Stripe was unsuccessfully being integrated and was not properly connecting to our application to process payments
- Solution: Failed to connect with because it did Azure did not have environment variables or other info

## Overall What We Learned

- Understanding of the full Software Cycle from design, architecture, functionality, implementation, testing, and maintenance
- How to work in a group and mastering the usage of Git,Github in a Team Environment
- Improving our programming skills and knowledge in React Native, Typescript, and Python
- Developing our knowledge to use foreign tools and software systems such as React Native, Django, Azure, MongoDB Atlas
- Enhancing our designing, testing, AI Integration skills including the complete understanding of how to integrate APIs
- How to use a Cloud service in a Full Stack Application

## What we have achieved!

- Core React Native frontend and Django backend setup
- MongoDB database integration
- User login and registration system
- Product browsing and category filtering
- Cart management and profile functionality
- RAWG API integration for live game data
- Dynamic checkout flow and order processing APIs
- Cloud-ready API architecture using Azure and MongoDB Atlas
- AI-powered game recommendations and semantic search
- AI-powered Chatbot





# References / Acknowledgements

## Acknowledgements

Instructor: Savita Mittal - Course Guidance and Project Direction

Team Members and Contributions:

- *Fazlul H Faizal- Owned System Architecture, AI Integration, Login/Register API Integration,*
- *Fotios Bampouridis- Owned Homepage, Search Page, Rawg API Integration, Email API Integration*
- *Moh Prajapati- Owned Frontend components, including Profile Page, Checkout Page, Chat Widget, Primary Document Contributor*
- *Joshua Delshad- Leader, Integrated Stripe, Integrated/Managed Azure Cloud, Integrated Database*
- *Eitan Abrishami – Owned Category Search Tab, Cart Operation, POST/GET Order API Integration, Order/User Database*

## Transparency and Support

All third-party APIs, libraries, and assets were used according to licensing and usage guidelines. Game data and metadata provided through the RAWG API comply with attribution requirements.

The team maintained collaborative development practices through GitHub version control, API testing, and continuous frontend/backend integration throughout Phase 2 development.